

connectivity. Even if maximum peer connections are reached, a network service should still accept and respond to a reasonable number of connections for the purposes of other node topology build-outs, etc.

Message delivery

The network delivers application messages, i.e., payloads received via *broadcast* or *send* to the application layer. The network also performs a basic validation of messages prior to forwarding by calling a handler in the Message Validation component.

When the network receives a *request* message, it calls a provided handler to determine if the request can be satisfied. If so, the expectation is that the application layer generates a *send* message with a response that satisfies the request. In this condition, the network layer does not continue to propagate the *request* message to the network.

In cases where a node could not satisfy the request, the node stores who it received the request from and *broadcasts* the request on to its peers. If that node receives a *send* message with the response to the request, it forwards the *send* message back to the original requester.

The network accepts application payloads for *BROADCAST*, *SEND*, and *REQUEST* from the application layer.

Network layer security

OMQ includes a TLS-like certificate exchange mechanism and protocol encryption capability that is transparent to the socket implementation. Support for socket-level encryption is currently implemented with server keys, which are read from the *validator.toml* configuration file. For each client, ephemeral certificates are generated on connect. If the server key pair is not configured, network communications between validators will not be authenticated or encrypted.

Network permissioning

The Sawtooth permissioning design allows the validator network to limit the nodes that are able to connect to it. The permissioning

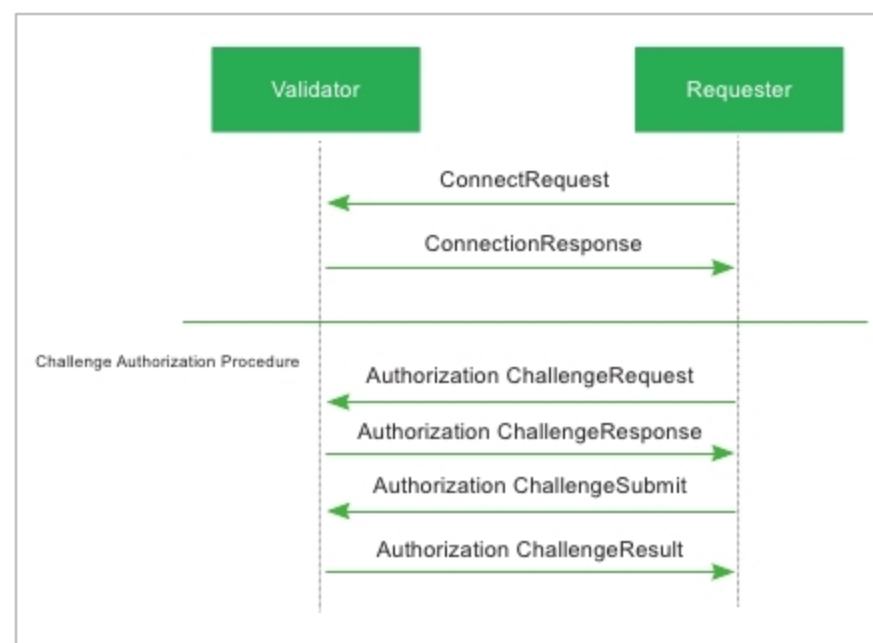


Figure 3: Trust authorisation

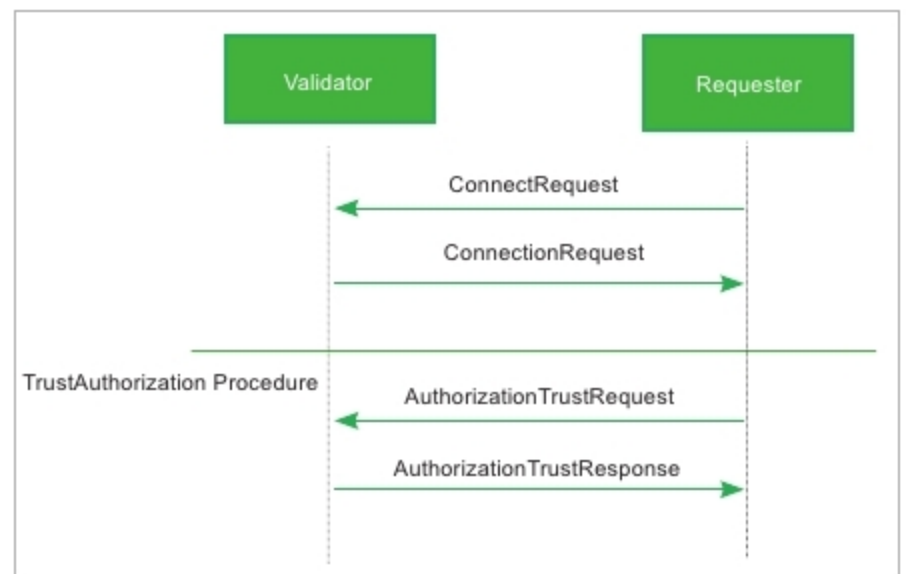


Figure 4: Challenge authorisation

rules determine the roles a connection is able to play on the network. The roles control the types of messages that can be sent and received over a given connection. Validators are able to determine whether messages delivered to them should be handled or dropped based on a set of roles and identities stored within the *Identity* namespace. Each requester is identified by the public key derived from its identity signing key. Permission verifiers examine incoming messages against the policy and the current configuration, and either permit, drop or respond with an error.

Authorisation types

Sawtooth implements two authorisation types: *trust* and *challenge*.

Trust is the simplest authorisation type. If trust authorisation is enabled, the validator will trust the connection and approve any roles requested that are available at that endpoint. If the requester wishes to gain access to every role it has permission to access, it can *request all* and the validator will respond with all available roles. However, if a role that is not available is requested, the requester is rejected and the connection will be closed. Figure 3 shows the message flow for trust authorisation.

If the connection wants to take on a role that requires a challenge to be signed, it will request the challenge by sending the empty *challenge* message to the validator that it wishes to connect to. The validator will send back a random payload that must be signed. The requester then signs the payload message and returns a response. The requester may also *request all*. The validator will respond with a status that says whether the challenge was accepted and the roles that the connection is allowed to take on. Figure 4 shows the message flow for challenge authorisation.

In the next article we will take a closer look at the Sawtooth permissioning design. **END** 🐧

By: Ashwini Kulkarni

The author is an independent consultant with prior experience in training, and actively follows emerging technologies like blockchain and containerisation. She's also certified in cyber law and cyber forensics.